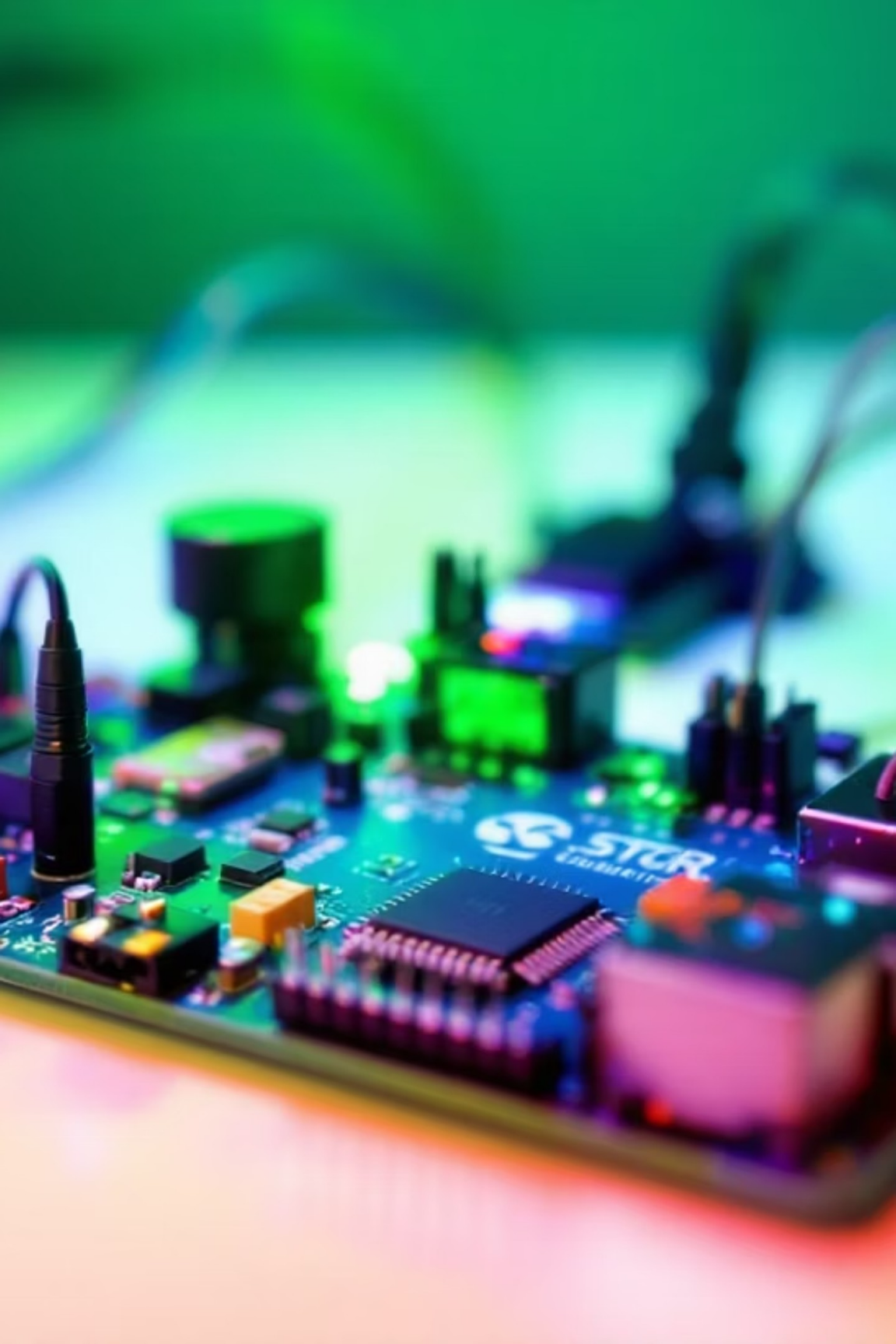




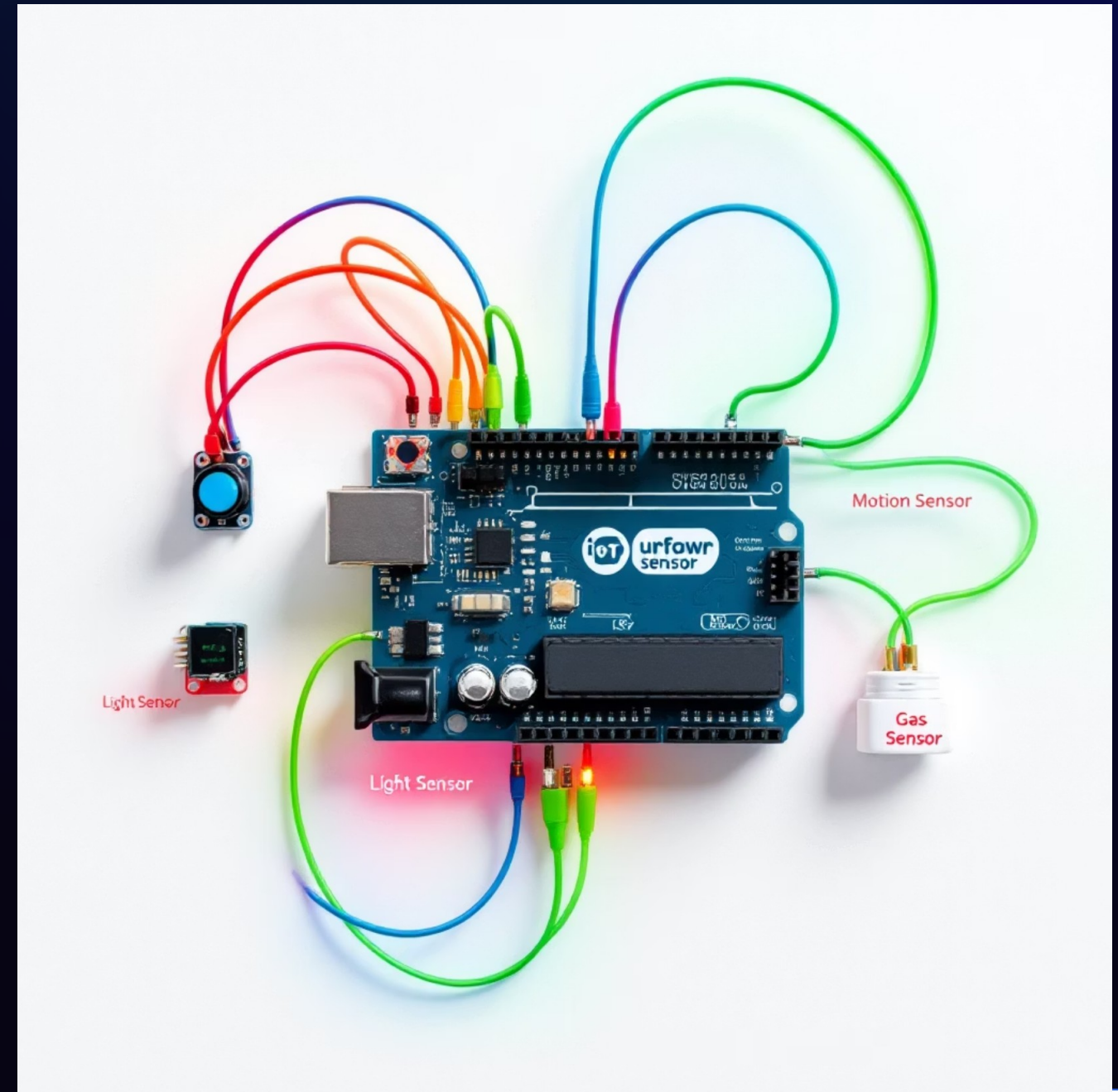
# Zephyr Workshop - Sensor Subsystem

A comprehensive introduction to Zephyr's sensor abstraction layer, featuring practical implementation with the TMP102 temperature sensor.

# What is the Sensor Subsystem?

The Sensor Subsystem in Zephyr RTOS provides:

- A generic interface to interact with various sensor hardware regardless of manufacturer
- Portable application code that works across different sensor implementations
- Unified API for accelerometers, gyroscopes, temperature sensors, and many more



# Why Abstraction Matters



## Vendor-Agnostic

Switch between sensor hardware without rewriting application logic, enabling easy hardware upgrades



## Simplified Architecture

Reduces complexity in the application layer by hiding implementation details



## Consistent Pattern

Universal workflow: fetch → get channel → process data

This abstraction is particularly valuable in embedded systems where hardware configurations change frequently during development.



# Sensor Driver API Overview



## **sensor\_sample\_fetch()**

Initiates a sensor reading operation, gathering fresh data from the hardware



## **sensor\_channel\_get()**

Retrieves specific channel values from the most recent fetch operation



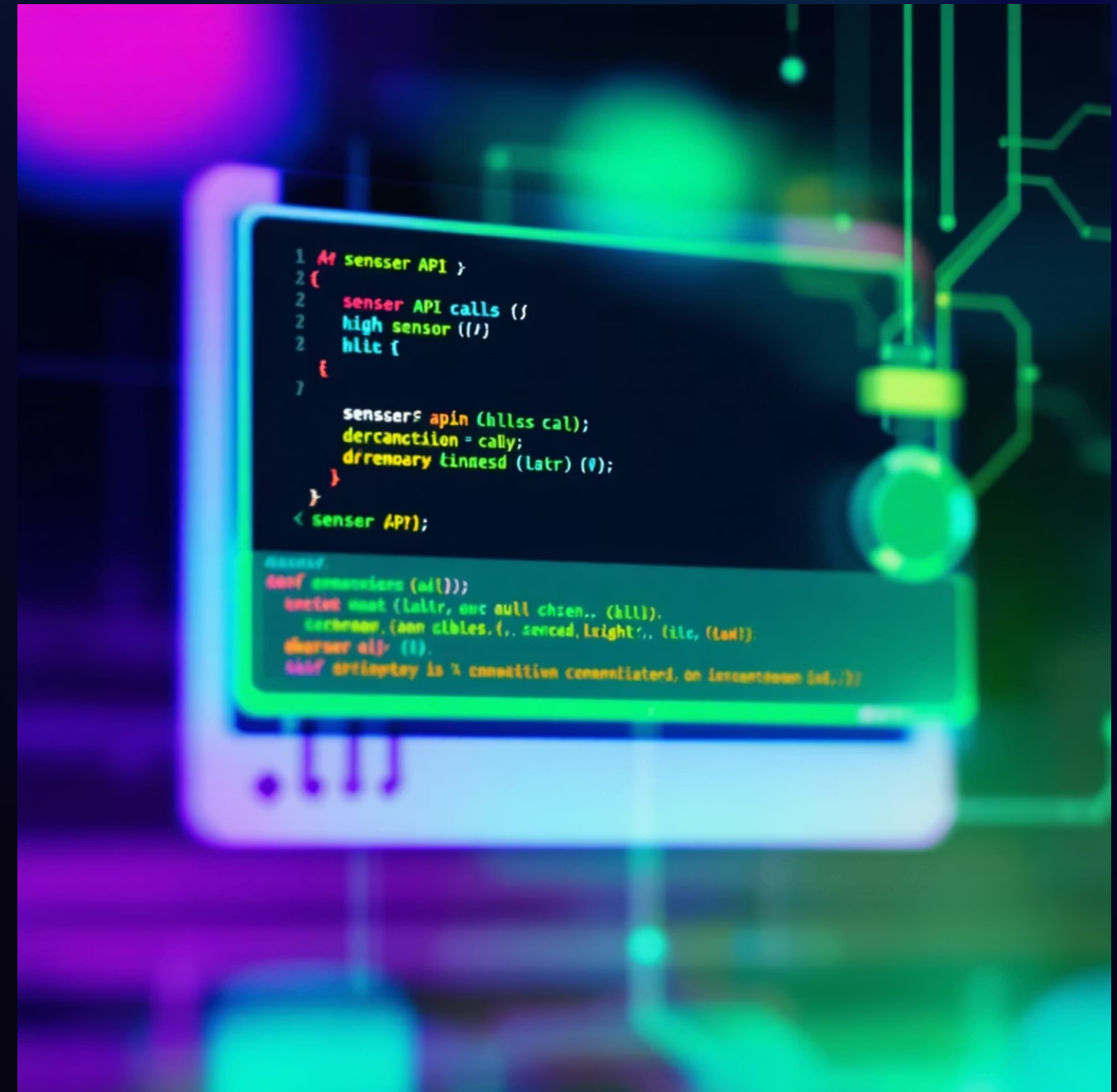
## **sensor\_attr\_set()**

Optional: Configure sensor attributes like scale, frequency, and sensitivity



## **sensor\_trigger\_set()**

Registers interrupt handlers or polling callbacks for sensor events



# Zephyr Sensor Channels

Zephyr defines standard channel types that represent different sensor measurements:

## **SENSOR\_CHAN\_ACCEL\_XYZ**

3-axis acceleration data,  
typically in  $\text{m/s}^2$

## **SENSOR\_CHAN\_GYRO\_XYZ**

3-axis angular velocity,  
typically in  $\text{rad/s}$

## **SENSOR\_CHAN\_TEMP**

Temperature readings in  
degrees Celsius

## **SENSOR\_CHAN\_LIGHT**

Ambient light levels, often in lux

## **SENSOR\_CHAN\_PRESS**

Atmospheric pressure, typically in kilopascals

Each channel type corresponds to a specific type of physical measurement that can be retrieved from compatible sensors.

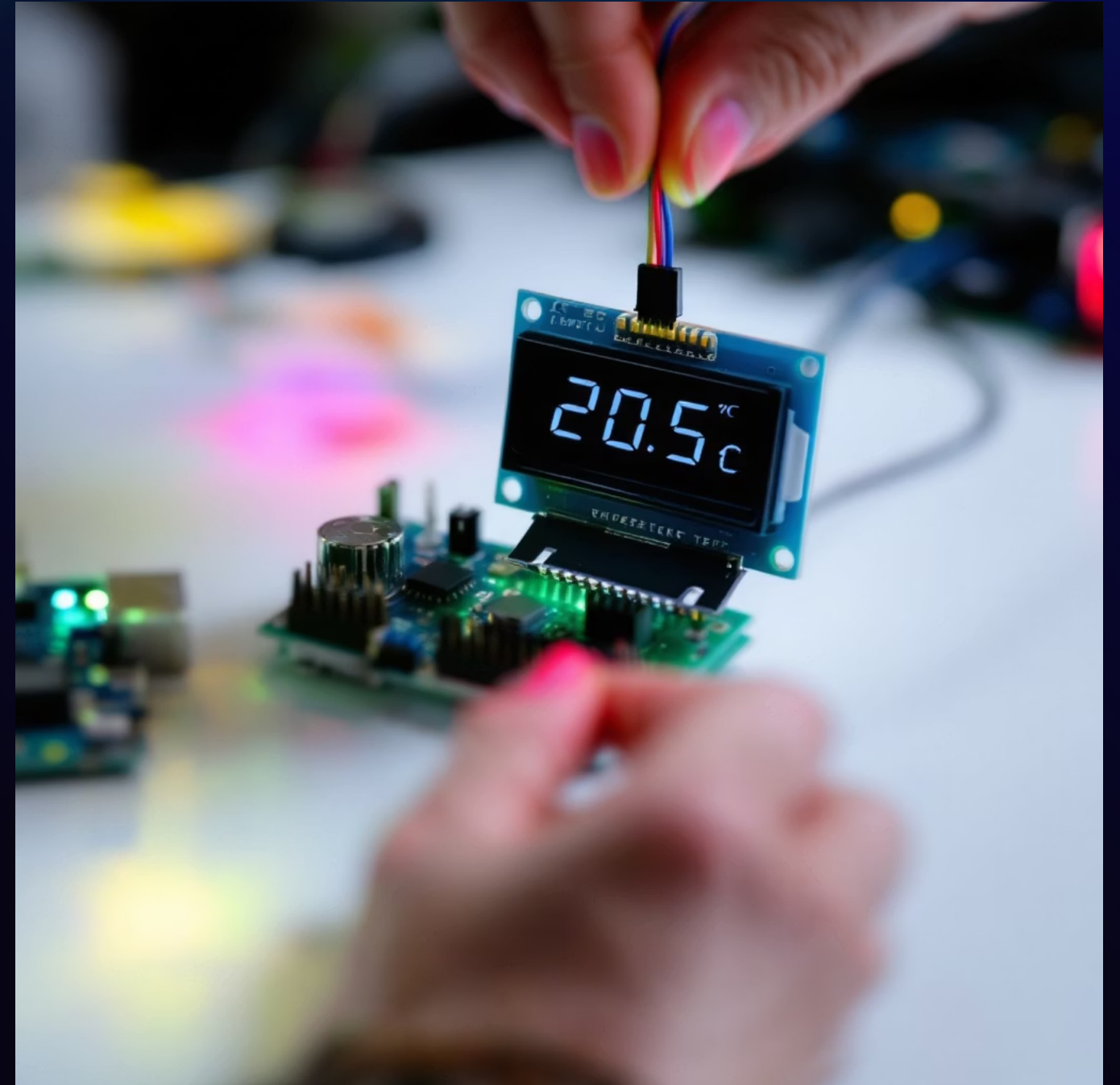
## Sensor Value Struct

```
struct sensor_value {  
    int val1;  /* integer part */  
    int val2;  /* fractional part in micro units  
*/  
};
```

For example, to represent 20.500000°C:

- val1 = 20 (integer part)
- val2 = 500000 (fractional part)

```
printf("%d.%06d", val.val1, val.val2);
```



This representation allows for precise readings while avoiding floating-point operations, which can be expensive on embedded systems.



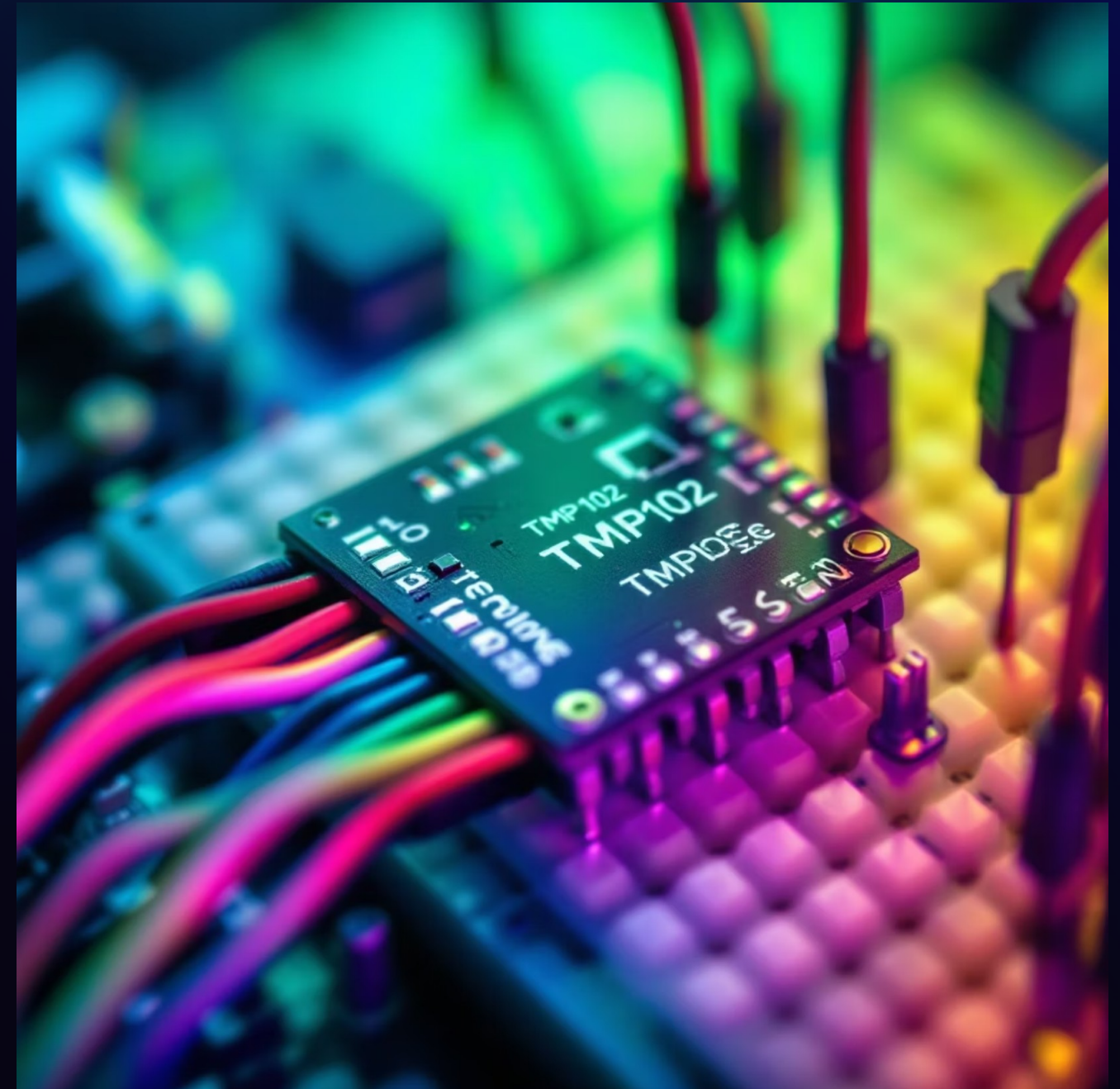
# TMP112 Implementation of Sensor API

The TMP112 temperature sensor demonstrates how to implement the Zephyr sensor API:

- Extends the generic sensor driver interface
- Implements required callbacks for initialization
- Provides channel-specific data retrieval
- Translates raw sensor values to standard format

Key files to examine:

- `include/zephyr/drivers/sensor.h` - API definition
- `drivers/sensor/tmp112/tmp112.c` - Implementation example



# What is Required in DeviceTree?

**1 Sensor Description**

Sensor must be properly described in the board's .dts file or in an application-specific overlay file

**2 Bus Connection**

Specify the connection method (typically I2C or SPI) with correct bus parameters

**3 Required Properties**

Include the "compatible" string to match driver, "reg" for address, and any GPIO interrupt connections



# Sample Demo: TMP102 Setup

To prepare the TMP102 demo:

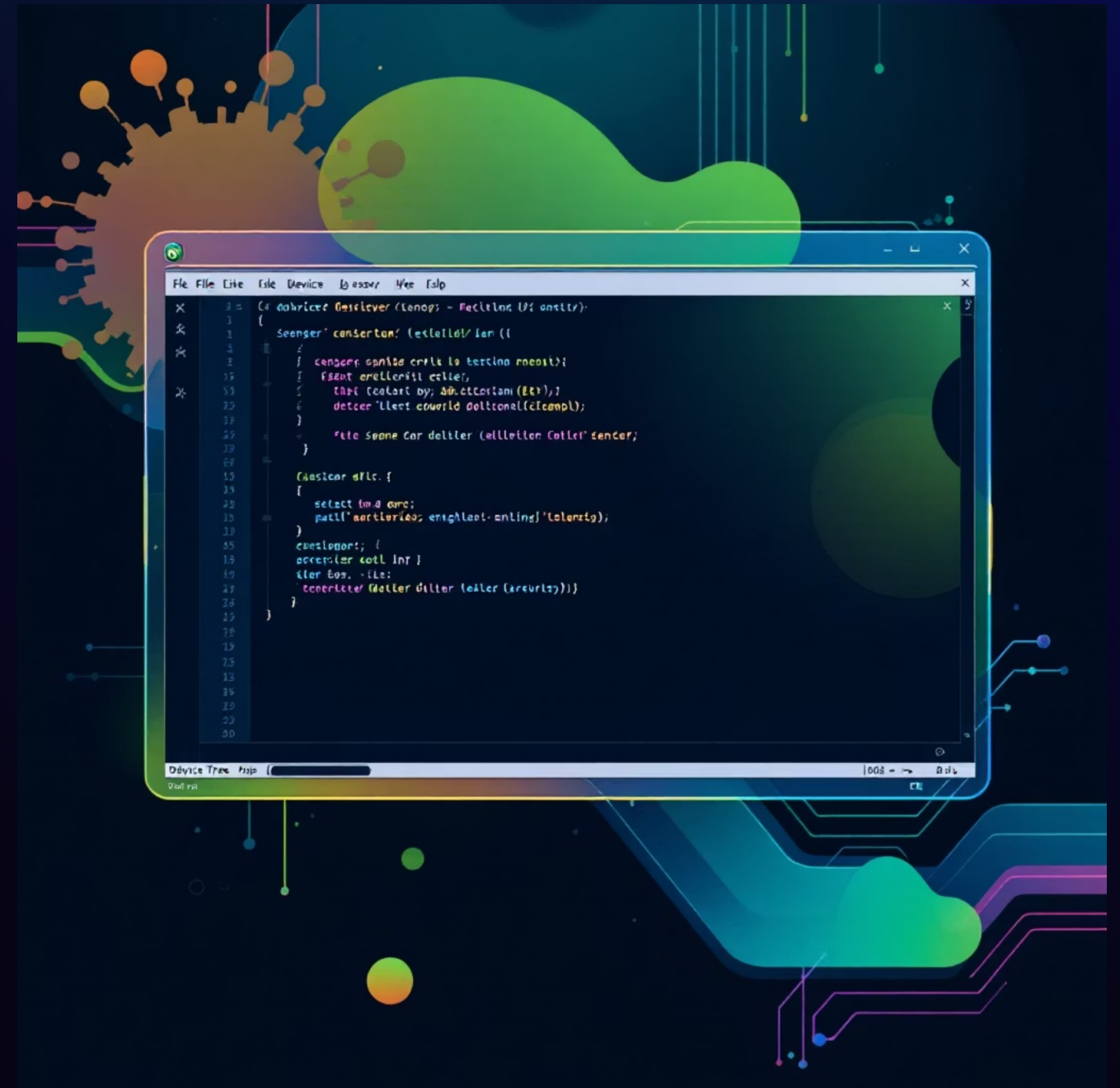
Navigate to the sample directory:

**`samples/sensor/tmp112`**

Create an **`app.overlay`** file for your board

3. \* Update the `arduino_i2c` node from `nrf52dk_nrf52832.dts`
4. \* Add the TMP102 sensor node with appropriate address

**Hint:** TMP102 is similar to TMP112 but uses a different I2C slave address. Check `frdm_k64f.overlay` for reference.



# Sample Demo: Building and Testing

## Build the Sample

Use the west build command to compile the sample for your target board:

```
west build -p always -b nrf52dk/nrf52832 samples/sensor/tmp112
```

## Connect Hardware

Wire the TMP102 sensor to the development board according to the pinout documentation.

## Flash and Test

Upload the firmware and verify operation:

```
west flash
```

Monitor the serial output to confirm temperature readings.

# Shell Integration

- Zephyr allows registering CLI commands via shell subsystem
- comment out the do\_main function
- add to prj.conf

```
CONFIG_SHELL=y CONFIG_SENSOR_SHELL=y
```

- rebuild and flash
- run 'sensor get tmp112@48 ambient\_temp'



# zephyr abstraction

- change zephyr board to nrf52833/40 or stm32wl
- rebuild and run